Jala University

# OdeolA

Oscar Steven Muñoz Fajardo

From a recorded meeting to useful information — automating audio processing and improving its performance step by step.

# Agenda

1. The problem and the solution
2. System architecture
3. Processing evolution
4. Benchmarks and evidence
5. Conclusion
6. Demo

# The problem

Recording a work meeting is easy.

But after the meeting, there is still important work: understanding what was said and what was agreed.

**Reviewing 45 minutes of audio to find the important points takes time.**

# The need

## Who does it affect?

People and teams that need to review recorded meetings without wasting time.

## What is lost today?

Time and important decisions can be lost when nobody has time to review the audio.

# The solution: OdeoIA

You upload the meeting audio and OdeoIA takes care of the rest: it reduces the noise, creates the transcription, creates a summary and leaves the audio ready to play.

# What is it built with?

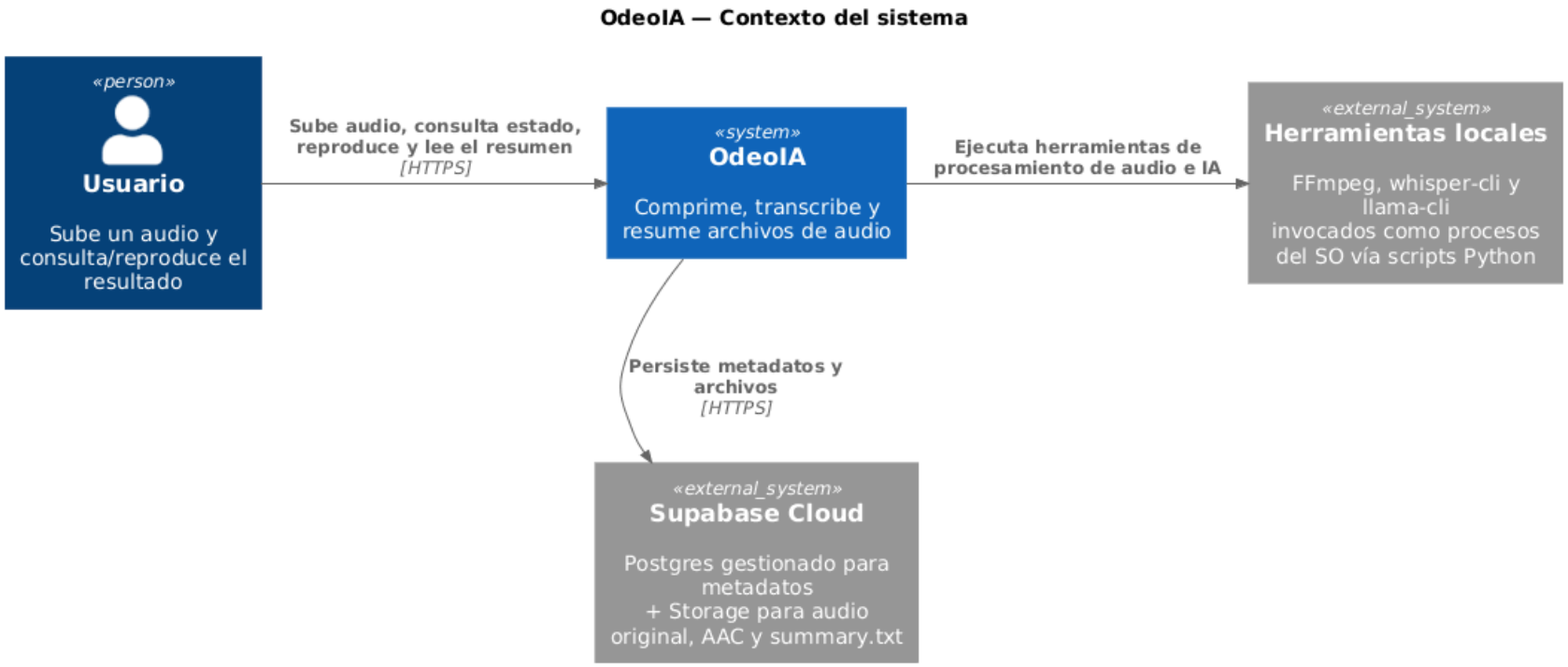
## Main technologies

- Backend: .NET 8
- Frontend: React + Vite + TypeScript
- Database: PostgreSQL
- Files: Supabase Storage
- Real-time status updates

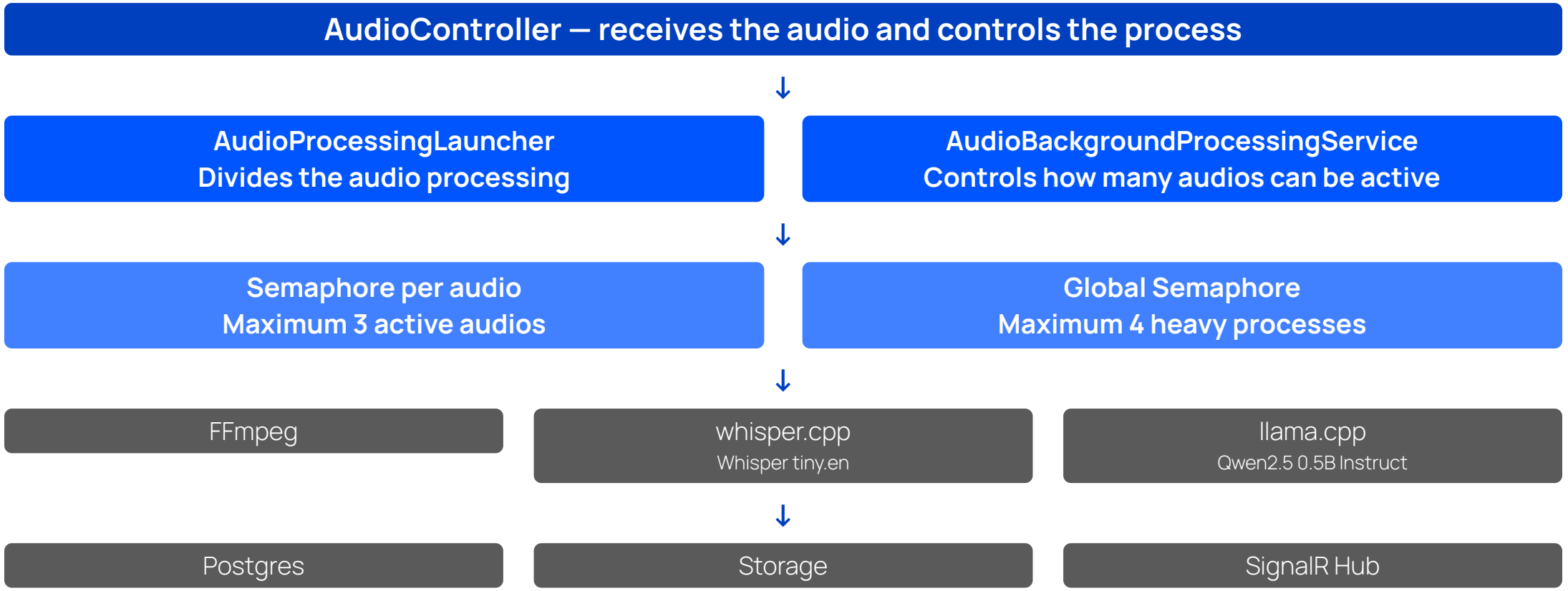
## Processing

- Audio: FFmpeg
- Noise filter: FFmpeg
- Transcription: whisper.cpp (Whisper tiny.en)
- Summary: llama.cpp (Qwen2.5 0.5B Instruct)
- Initial audio validation in the browser

# Architecture — Context



# Architecture — Components



# Processing pipeline

**Client uploads the audio**

↓

**Flow 1 — Audio processing**

Compression (FFmpeg)

↓

Noise filter (afftdn)

**Flow 2 — Transcription and summary**

Transcription (whisper.cpp)

↓

Summary (llama.cpp)

These tasks can make progress at the same time

The global Semaphore limits heavy processes, even when several audios are active.

↓

**The results are saved and the status is updated in real time**

# How the solution evolved




Each stage appeared to solve a problem I found in the previous one — including `Parallel.ForEachAsync` itself, which was eventually replaced in the final version.

# Stage 1 — First version with Threads



**First version**

The first version used one `Thread` for each flow: one for audio processing and another for transcription and summary.

**Problem:** there was no clear control over how much work could run at the same time.

```
var thread = new Thread(() =>
{
    // Runs one of the processing flows.
    //
    invoke(handler, audioRecordId,
        correlationId)
        .GetAwaiter().GetResult();
});

thread.Start();
```

# Stage 2 — Split the flow

**ThreadPool / TPL**

I realized that audio processing and transcription did not directly depend on each other, so I separated them into two tasks that could move at the same time.

**Improvement:** I no longer had to manage Threads directly, and I could make better use of independent tasks.

```
Task compressionTask = Task.Run(() =>
    RunFlowAsync<ProcessAudioCompression>(
        "Compression", audioRecordId, ...));

Task summaryTask = Task.Run(() =>
    RunFlowAsync<ProcessAudioSummary>(
        "Transcription+Summary", id, ...));

await Task.WhenAll(
    compressionTask, summaryTask);
```

# Stage 3 — Control concurrency

## Semaphore

After allowing more tasks at the same time, another problem appeared: several heavy operations could run together and use too many resources.

I used a global `SemaphoreSlim` to limit how many heavy operations can run at the same time. In the final configuration, the limit is 4.

```
await _externalProcessSemaphore
    .WaitAsync(cancellationToken);

try
{
    // Heavy process:
    // FFmpeg / whisper / llama
}
finally
{
    _externalProcessSemaphore.Release();
}
```

# Stage 4 — Parallel.ForEachAsync



**Parallel / Parallel programming**

There was still another problem to solve. I not only wanted to process parts of one audio at the same time, I also wanted to receive several audios without losing control.

I used `Parallel.ForEachAsync` to allow up to 3 audios at the same time, taking items from the queue in groups of up to three.

```
var options = new ParallelOptions
{
    MaxDegreeOfParallelism = maxDop,
    CancellationToken = stoppingToken,
};

await Parallel.ForEachAsync(
    claimedIds, options,
    async (audioId, ct) =>
{
    await launcher.ProcessAsync(
        audioId, ..., ct);
});
```

# Batches vs. slots: the key difference

## Parallel.ForEachAsync — by batches

Audio 1 | Audio 2 ✓ | Audio 3

Audio 2 finishes, but its place remains unused

↓

Wait for 1 and 3 to also finish

↓

**Only then does a new batch enter**

## Final Semaphore — by slots

Slot 1: Audio 1 | Slot 2: Audio 2 ✓ | Slot 3: Audio 3

Audio 2 frees its slot when it finishes

↓

Slot 2 is available immediately

↓

**Audio 4 enters immediately — without waiting for 1 or 3**

# Stage 5 — Final architecture: Semaphore per audio

## Final architecture

I replaced `Parallel.ForEachAsync` with a `SemaphoreSlim` with individual slots for audios. The current limit is 3 active audios.

This control does not replace the global Semaphore: they work at different levels. The audio one controls how many audios are active, and the global one limits heavy processes to 4.

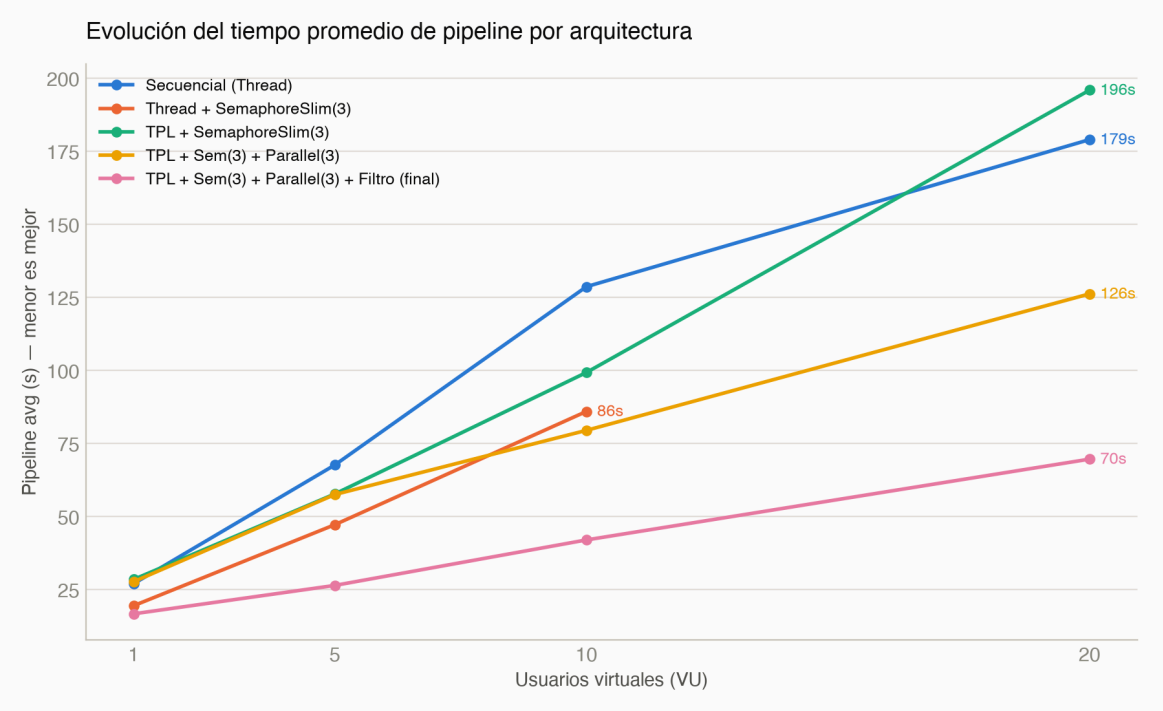
**This is the version currently running in production.**

```
foreach (var audioId in claimedIds)
{
    // Slot for the complete audio.
    await _slots.WaitAsync(
        stoppingToken);

    Dispatch(audioId);
}

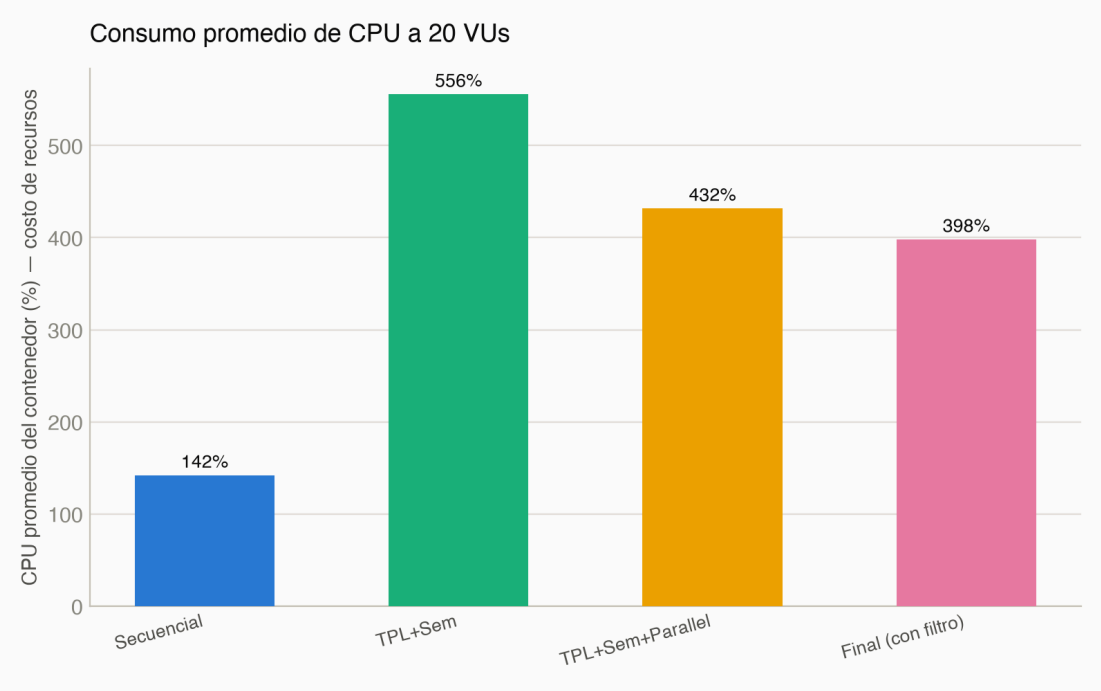
// Inside each flow:
// Global Semaphore = maximum 4
// heavy processes.
```

# Benchmarks — did it really improve?




At 20 virtual users: 179s in the first version → 70s in the final version. An important result: TPL + Semaphore without Parallel reached 196s, even worse than the first version.

# Benchmarks — resource usage per architecture




TPL + Semaphore without real parallelism used the most CPU (556%) despite being one of the slowest configurations — using more CPU does not automatically mean processing faster.

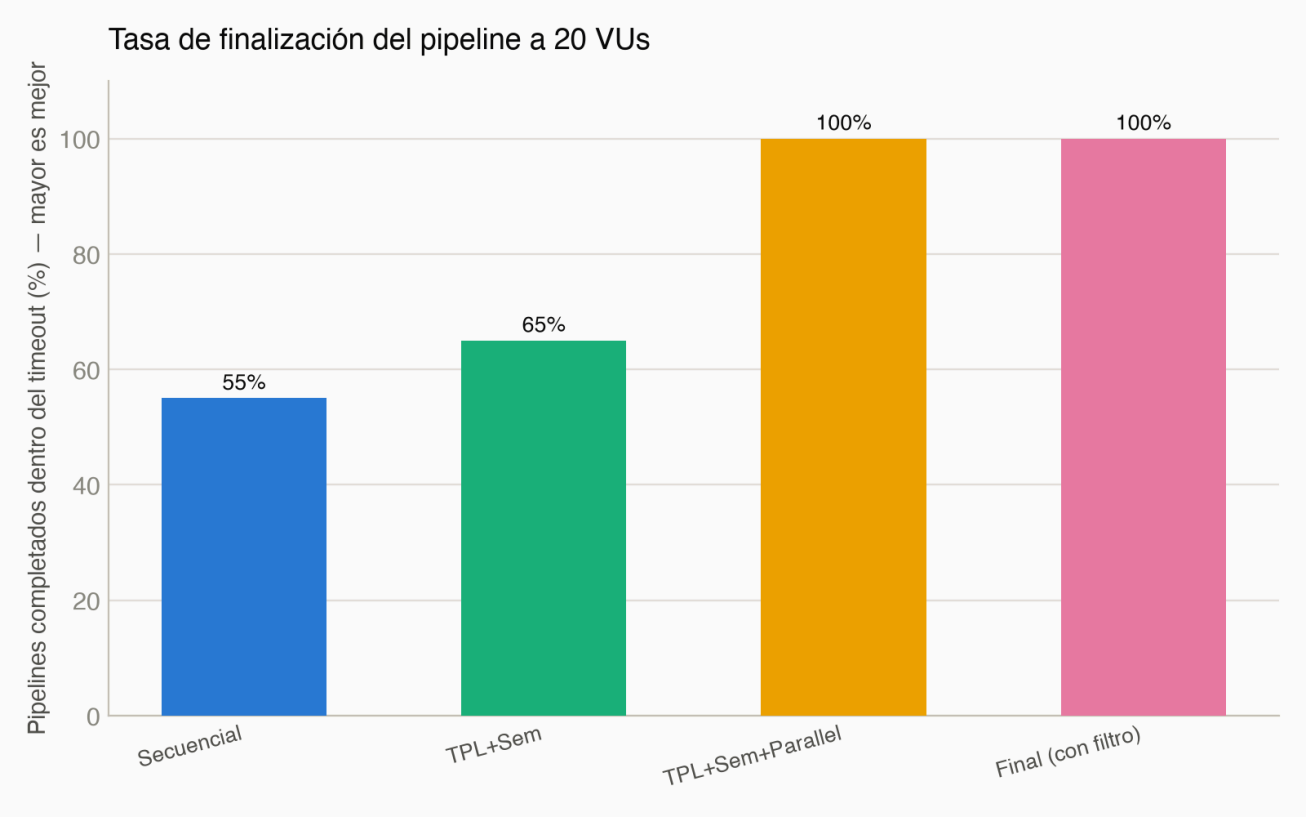
# How long does each stage take, per architecture?

| Architecture (20 VU) | Compression | Transcription | Summary | Pipeline avg |
|---|---|---|---|---|
| Sequential | 2.1 s | 5.2 s | 9.6 s | 179.1 s |
| TPL + Semaphore(3) + Parallel.ForEachAsync(3) | 2.1 s | 5.0 s | 10.5 s | 126.3 s |
| **Final Semaphore (audios)** | **1.5 s** | **4.2 s** | **8.9 s** | **68.4 s** |

Stage duration was not captured for the TPL + Semaphore run — that benchmark predates per-stage metric collection in the script, so only its total pipeline time is available.

**Each stage takes a similar amount of time across architectures (a few seconds each) — the big difference in total pipeline time does not come from the stages themselves, it comes from how long an audio waits before it is even admitted for processing.**

,




Tasks completed at 20 VU

Each k6 virtual user waits at most 4 minutes (240s) for its audio to reach `Completed` — that is why the sequential version only shows 55% completion at 20 VU: those audios did not fail, they simply did not finish within that window.

# Why this final architecture?

| Stage | Solved | Limitation |
|---|---|---|
| Threads | Basic concurrency | No control over how many threads are created |
| TPL | Real parallelism within one audio | Does not limit how many audios run at once |
| Semaphore | Protects CPU/RAM from FFmpeg/whisper/llama | Does not limit how many complete audios are processed |
| Parallel.ForEachAsync | Several audios at the same time | By batches — a slow audio blocks free slots |
| **Semaphore final** | **Controls audios by slots** | Does not remove the global process limit |

**I did not choose it because it was theoretically better: with the same 100% completion rate, it reduced total time by 33.7% and lowered CPU use. Profiling also showed that it was important to control audio admission without depending on batches.**

# Conclusion



- **Initial problem:** processing did not respond well when the load increased.
- **Evolution:** Threads → TPL → Semaphore → Parallel.ForEachAsync → Semaphore final por audio.
- **Applied concepts:** Threads, tasks, async/await, ThreadPool, Semaphore and Parallel — including why `Parallel` was replaced.
- **Result:** ~50% less time under load (179s → 70s at 20 VU) and from 55% to 100% of completed tasks.
- **Final approach:** Semaphore per audio — same success rate as Parallel.ForEachAsync, but 33.7% faster and with less CPU.
- **Key lesson:** doing more things at the same time does not always mean doing it faster, and not every parallelism mechanism behaves the same under uneven load.

Jala University

# Demo

OdeoIA in action

# Backup — Simplified request sequence

1. Client makes `POST /api/audio/upload` with the file.
2. The controller saves the original audio and marks it `Pending`.
3. `AudioProcessingLauncher` downloads the audio once and starts Compression and Transcription → Summary as independent `Tasks`.
4. Each flow acquires the global `SemaphoreSlim` before calling FFmpeg / whisper.cpp / llama.cpp, with a maximum of 4 heavy processes at the same time, and releases it when finished.
5. Audio processing uses a maximum of 3 slots; an audio can occupy a slot while waiting for the global limit to be available.
6. Each stage saves its result and updates the status.
7. SignalR notifies the client in real time — without polling.
8. The client plays the filtered audio and reads the summary.

# Backup — Next steps

- Consolidate benchmark results into a single CSV for historical analysis between variants.
- Live metrics dashboard (duration per stage, CPU, success rate) instead of reports per run.
- Explore moving the noise filter before compression to evaluate its impact on perceived quality.
- Auto-scale `MaxDegreeOfParallelism` based on available CPU instead of a fixed value per environment.